Simple process for sharing Rmarkdown code via Docker

now

Introduction

Lets assume you have an rmarkdown `Rmd` file, say `peng.Rmd`, that you're written to analyze some data. You now want to share the code with a colleague, we'll call him Joe. How to proceed?

The simplest option is simply to send Joe the "rmd" file containing the code via the most convenient method (e.g. email/text/slack/discord/github/USB drive etc.)

The next step will be for the Joe to (attempt to) load and run the code. Typically he would do this with either using `Rstudio.app` to open the file and `knit` it, render it from the command line with the command:

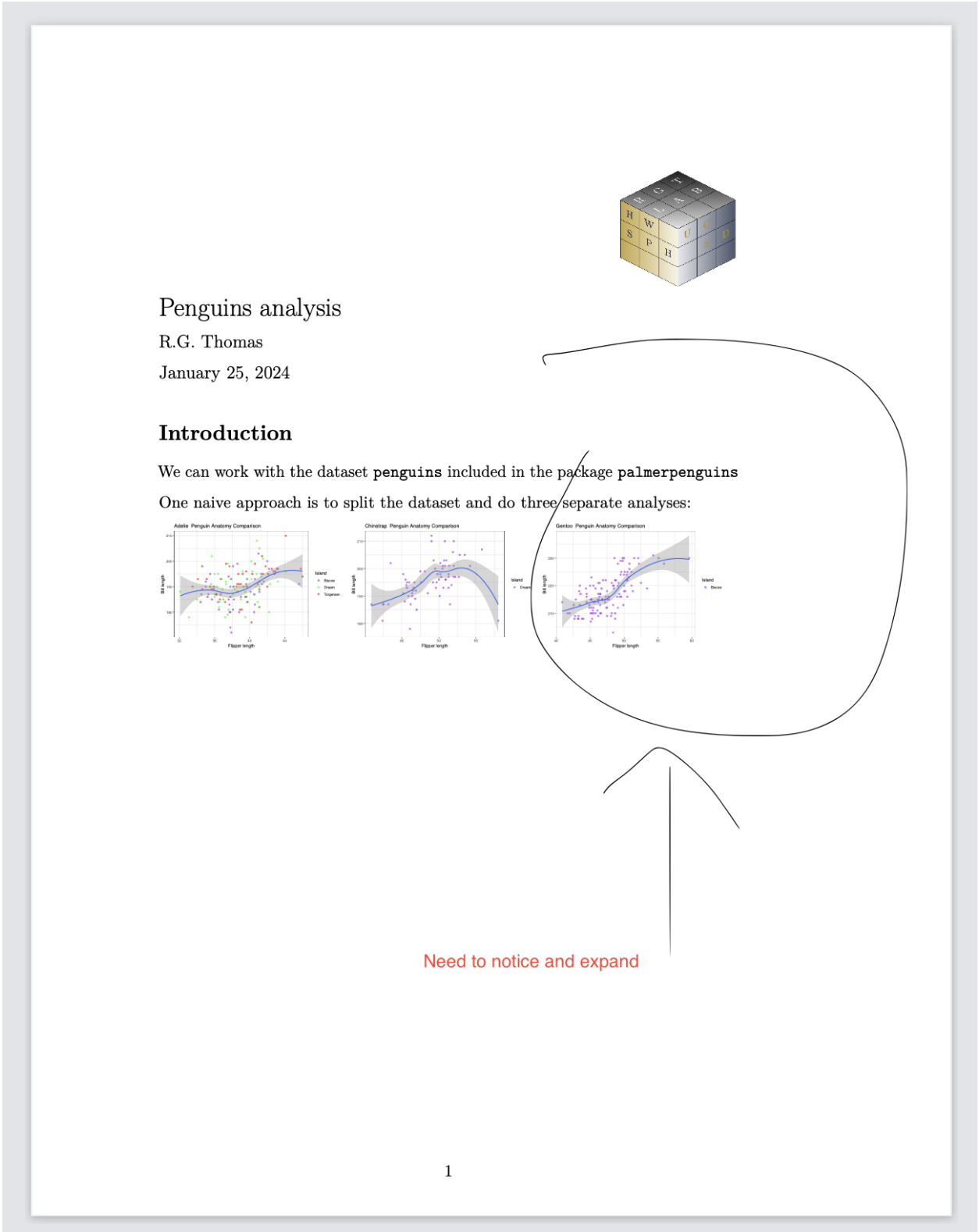
```
> R -e "render('peng.Rmd')"
```

Sometimes this approach works, and all is well. Joe can add comments or expand the code and reply to you. Frequently, however, this naive process will fail for any number of reasons. Ideally to facilitate reproducibility Joe will have as similar a computing environment as you, the original developer. This can be difficult to achieve, especially given the dynamic nature of open source software. For example Joe may have an outdated version of `R` installed on his workstation, or his `R` environment may be missing a necessary package. Additiional potential problems include: the required package may be present but its the wrong version, the program may need to source an additional file thats missing, or the program load some data that it can't find on Joe's machine.

All of these problems go away if instead of sending the program as a standalone text file you send it as a docker image. In this post we'll walk through the process of dockerizing the R code.

Assume a simple `Rmd` file like this:

The `Rmd` file runs cleanly on our machine and generates the the report on the following page. However, we note that the third plot needs additional examination and want to relay the program to our colleague Joe for further analyis.



Penguins analysis

R.G. Thomas

January 25, 2024

Introduction

We can work with the dataset **`penguins`** included in the package **`palmerpenguins`**

One naive approach is to split the dataset and do three separate analyses:

Need to notice and expand

1

Figure 1: rendered page

Share program code with Joe. Two approaches, Naive and Docker based.

Whats the best way to accomplish this?

We start by simply emailing the file to him (rgthomas4747@gmail.com) and asking him to collaborate.

Joe downloads the attachment. Opens a working directory and attempts to run the Rmd file

with the command

```
> R -e "render('peng.Rmd')"
```

Joe has a linux mint desktop

```
> mkdir peng_collaboration
> cd peng_collaboration
> R -e "render('peng.Rmd')"
```

Linux can'f find R

Joe can fix this by installing `R`

```
> sudo apt install r-base-core
```

Next `R` can not find the function `render`.

Joe determines that `render` is a function in the package `rmarkdown`

He endeavors to installs `rmarkdown` with

```
R -e "install.packages('rmarkdown')"
```

This fails due to inadequate permission on the directory `/usr/lib/R/library`

```
sudo apt install libssl-dev libcurl4-openssl-dev unixodbc-dev libxml2-dev\
libmariadb-dev libfontconfig1-dev libharfbuzz-dev libfribidi-dev\
libfreetype6-dev libpng-dev libtiff5-dev libjpeg-dev
```

Also latex is not available

One more try... and the latex engine notes the absence of the file

```
~/shr/preamble.tex
```

So, I need to relay the missing .tex file.

Also the .png (sudoku.png) logo file.

Finally! success.

Docker approach

Alternatively, consider the "Docker" approach.

Before sending peng.Rmd to Joe we'll dockerize it.

- Prepare a work directory: penguins. We want to send Joe a container that has R and all the preliminaries taken care of so that all he has to do is

Here is the docker file

```
FROM rocker/verse:4
RUN apt update
RUN apt install vim -y
RUN R -e "install.packages('pacman')"
RUN R -e "install.packages('palmerpenguins')"
RUN R -e "install.packages('tidyverse')"
RUN R -e "install.packages('knitr')"
RUN R -e "install.packages('rmarkdown')"
RUN tlmgr init-usertree
RUN tlmgr update --self --all
RUN  tlmgr install  fancyhdr adjustbox geometry titling

RUN addgroup --system joe && adduser --system --ingroup joe joe
RUN chmod -R 0777 '/usr/local/lib/R/site-library'
RUN chown joe:joe -R /home/joe
USER joe
WORKDIR /home/joe
RUN mkdir -p /home/joe/shr
RUN mkdir -p /home/joe/output
COPY /preamble.tex /home/joe/shr
# COPY /.Rprofile /home/joe/shr
COPY sudoku.png /home/joe/shr
COPY peng.Rmd /home/joe/shr
CMD ["/bin/bash"]
```

run docker

```
docker build -t rgt47/penguin_review --platform=linux/amd64 .
docker push rgt47/peng_review
```

relay image to Joe

```
docker push rgt47/peng_review
```

or

```
docker save rgt47/peng_review | gzip > peng_review_trans.tgz
docker load -i peng_review_trans.tgz
```

```
> docker pull rgt47/penguin_review

> droot="$PWD"/output docker run -it --rm --platform linux/x86_64 \
-v $droot:/home/joe/output peng_review
> cd output
> library(rmarkdown); render('../shr/peng.Rmd')
```

Important to include the association between the /home/joe/output directory in the container with the output directory on the local workstation. Thats where the results of the analysis will be saved.

```
> R -e "library(rmarkdown); render('peng.Rmd')"
```

and if he wants to edit peng.Rmd

```
> vim peng.Rmd
```

```
\usepackage[export]{adjustbox}
\usepackage{fancyhdr}
```

```
\usepackage{titling}

\pagestyle{fancy}

\pretitle{
\begin{flushright}
\includegraphics[width=3cm,valign=c]{sudoku.png}\\
\end{flushright}
\begin{flushleft} \LARGE }
\posttitle{\par\end{flushleft}\vskip 0.5em}
\predate{\begin{flushleft}\large}
    \postdate{\par\end{flushleft}}
    \preauthor{\begin{flushleft}\large}
    \postauthor{\par\end{flushleft}}
\fancyfoot[L]{\currfilename} %put date in header
\fancyfoot[R]{\includegraphics[width=.8cm]{sudoku.png}}
\fancyhead[L]{\today} %put current file in footer
```

REFERENCES

Running your R script in Docker

Practice

Notes.

Situation: Mint is loaded and peng1.Rmd is transferred from researcher 1 to /home/z directory.

Try to run the program with command from colleague:

> R -e "render('peng1.Rmd')"

Result: error message:

Command R not found.

So install R.... to fix

> sudo apt install r-base-core Try again...
>
> R -e "render('peng1.Rmd')" Result: R loads but gives error: could not find function "render"

Search google "R render"

Result: https://pkgs.rstudio.com/rmarkdown/reference/render.html

Looks like render is a function inside rmarkdown package. so install package R> install.packages("rmarkdown")

Try again.

> R -e "library(rmarkdown); render('peng1.Rmd')"

Result: error. pandoc version 1.12.3 or higher required.

Now try to install pandoc Try again. > R -e "library(rmarkdown); render('peng1.Rmd')" Result: error. there is no called pacman

so install pacman R> install.packages("pacman") Try again...

> R -e "library(rmarkdown); render('peng1.Rmd')" result: failed pandoc could not find /Users/zenn/shr/preamble.tex

This makes sense we forgot to send the preamble.tex file to researcher 2. Lets send it now. Also we need to adjust its location from a macos style file name to a linux based one.

To edit peng1.Rmd we need vim > sudo apt install vim

Try again Result: pandoc error "pdflatex not found" Lets install tinytex. First in R and then in mint R> install.packages("tinytex") R> tinytex::install_tinytex() # to uninstall TinyTeX, run # tinytex::uninstall_tinytex()

Try again:

> R -e "library(rmarkdown); render('peng1.Rmd')" result: pandoc error: file sudoku_apple.pdf not found.

This makes sense we forgot to send the logo file. Lets send the file and try again.

> R -e "library(rmarkdown); render('peng1.Rmd')" Result: error no bibliography file found.

This makes sense we forgot to send the bib file. Lets send the file and try again. Also need to edit the location of bib file.

Try again:

> R -e "library(rmarkdown); render('peng1.Rmd')"

results: minor error. Some packages weren't loading via pacman.

Try removing janitor, kableExtra, tidyverse, readxl and add ggplot2.

Try again:

> R -e "library(rmarkdown); render('peng1.Rmd')"

Success!